

Implementación de un Agente A*

En este ejercicio vas a implementar un agente de **búsqueda informada** sobre el framework `pathFinding`: un entorno de laberintos generados proceduralmente en el que un agente debe desplazarse desde una celda de inicio hasta una celda objetivo. A diferencia de los agentes reflex vistos en la práctica del Aspirador, aquí el agente sí puede planificar: recibe el laberinto completo y debe encontrar el camino de **menor coste** hasta el objetivo.

El algoritmo que vas a implementar es **A***. A* mantiene una frontera de nodos candidatos ordenada por

$$f(n) = g(n) + h(n),$$

donde $g(n)$ es el coste real acumulado desde el inicio hasta el nodo n , y $h(n)$ es una **heurística admisible** que estima (sin nunca sobrestimar) el coste restante hasta el objetivo. En cada paso, A* expande el nodo de la frontera con menor $f(n)$; si la heurística es admisible, el camino que devuelve es óptimo.

No modifiques ningún archivo dentro de `utils/maze_core/`.

Instalación y puesta en marcha

El proyecto vive en su propio repositorio. Sigue estos pasos para clonarlo e instalar el entorno:

```
git clone https://github.com/alvarotorijano/pathFinding
cd pathFinding

python -m venv venv
source venv/bin/activate # En Windows: venv\Scripts\activate

pip install -r requirements.txt
```

El framework funciona completamente sin ningún paso adicional. Existe un solver de referencia privado (submódulo `utils/labyrinth-solver-wasm`) que el profesorado puede instalar para mostrar el coste óptimo real; si no está instalado, las métricas `Optimal Cost` y `Optimality` de la salida aparecerán a 0.0 sin que eso afecte a tu entrega.

Comprueba que todo funciona generando un laberinto y ejecutando uno de los agentes de ejemplo ya incluidos (`random_agent`, `dfs_agent`):

```
python -m labyrinth generate --width 20 --height 20 --type perfect --seed 42 --output
maze.maze

python -m labyrinth run --agent dfs_agent --maze-file maze.maze --visualize --speed 5
```

Deberías ver una animación del agente recorriendo el laberinto y, al final, un resumen con métricas (`Steps`, `Path Cost`, `Nodes Explored`, ...).

El entorno

Cada agente hereda de `Agent` (`utils/maze_core/agent.py`) e implementa un único método, `step`, que recibe una observación y devuelve una dirección de movimiento. El estado que guardes en `self` **persiste entre llamadas**, lo cual es imprescindible para A*: solo hace falta ejecutar la búsqueda completa la primera vez que ves un laberinto y después ir devolviendo, paso a paso, las direcciones del camino ya calculado.

Percepto (Observation):

Clave	Tipo	Significado
maze	Maze	Laberinto completo: topología y coste de cada celda (<code>maze.get_neighbors(pos)</code> , <code>maze.get_cell(pos).cost</code>).
current	Position	Posición actual del agente.
goal	Position	Posición objetivo.
step_count	int	Número de movimientos realizados hasta ahora.
path_cost	float	Coste acumulado hasta ahora.

Acciones: `Direction.NORTH`, `SOUTH`, `EAST`, `WEST` (solo son válidas las direcciones devueltas por `observation.maze.get_neighbors(observation.current)`).

Memoria: el agente *necesita* memoria entre pasos. Como mínimo deberás guardar: una **lista** de posiciones candidatas (`self.frontier`) con las celdas que aún quedan por explorar, un diccionario `self.g_score` con el coste acumulado real hasta cada celda visitada, un diccionario `self.parent` para reconstruir el camino, y el propio camino ya calculado en `self.path` (para ir devolviéndolo paso a paso sin repetir la búsqueda en cada llamada a `step`). En cada iteración, el siguiente nodo a explorar es el de `self.frontier` con menor $f(n) = g(n) + h(n)$: no hace falta ninguna estructura especial de cola de prioridad, con `min()` sobre la lista es suficiente.

Métodos y funciones que te van a ser útiles:

Función	Qué hace
<code>maze.get_neighbors(pos)</code>	Lista de <code>Direction</code> válidas desde <code>pos</code> (según los muros del laberinto).
<code>pos.move(direction)</code>	<code>Position</code> resultante de moverte una celda en esa dirección.
<code>maze.get_cell(pos).cost</code>	Coste (<code>float</code>) de entrar en la celda <code>pos</code> .
<code>pos.distance_to(goal)</code>	Distancia Manhattan entre dos posiciones (ya usada dentro de <code>_heuristic</code>).
<code>min(lista, key=...)</code>	Built-in de Python: devuelve el elemento de lista que minimiza <code>key(elemento)</code> . Úsalo para elegir, en cada iteración, el nodo de <code>self.frontier</code> con menor $f(n)$.
<code>diccionario.get(clave, valor_defecto)</code>	Devuelve <code>diccionario[clave]</code> si existe, o <code>valor_defecto</code> si no. Útil para consultar <code>self.g_score</code> de una celda que quizá aún no tenga coste asignado.

Se pide:

- a) Implementa el agente A* completo en `agents/a_star_agent/agent.py` (plantilla ya presente en el repositorio, con la misma estructura que el agente `dfs_agent` ya implementado: inicialización al ver un laberinto nuevo, bloque para seguir el camino ya calculado, y al final el bloque a rellenar, marcado como `YOUR CODE GOES HERE`, donde va la búsqueda). Debes:
 - (I) Mantener `self.frontier` como lista de posiciones candidatas, seleccionando en cada iteración la de menor $f(n) = g(n) + h(n)$ (por ejemplo, con `min()`).
 - (II) Usar como heurística $h(n)$ la distancia Manhattan hasta el objetivo (ya implementada en `_heuristic`, no hace falta que la modifiques).
 - (III) Tener en cuenta el coste real de cada celda (`maze.get_cell(vecino).cost`), no solo el número de pasos.
 - (IV) Reconstruir y almacenar el camino óptimo encontrado (`self.path`) la primera vez que el agente ve un laberinto nuevo.
- b) Justifica en 3–4 líneas (en el propio `README.md` del agente o en un comentario al principio de `agent.py`) por qué la distancia Manhattan es una heurística admisible en este entorno, y qué ocurriría si los laberintos permitiesen movimientos en diagonal.
- c) Ejecuta tu agente sobre al menos **dos laberintos distintos** (por ejemplo, tamaños o `-type` distintos: `perfect`, `multiple`, con `-cost-map random` o `heatmap`) y compara sus métricas de salida (Steps, Path Cost, Nodes Explored) frente a `dfs_agent` y `random_agent` en los mismos laberintos.

Entrega:

Un único archivo: `agents/a_star_agent/agent.py` con tu implementación de A*. Puedes acompañarlo, opcionalmente, de un `README.md` en la misma carpeta con la justificación teórica del punto (b) y las comparativas del punto (c).